TaskBoard Lite

End-to-end DevOps project: code to production with CI/CD, DevSecOps, Kubernetes, Terraform, monitoring and GitOps

Vivek Singh Solanki | Roll No. 24bcs10338 | Session 21 - Final DevOps Project & Troubleshooting

github.com/vivek-0509/Devops_Final_Assignment_N_Project

What this presentation covers

1

Goal & architecture

What TaskBoard Lite is and how every DevOps layer fits together

2

Application & Docker

Flask API, tests, multi-stage Alpine image, docker compose

3

CI/CD & DevSecOps

GitHub Actions pipeline with SAST, SCA, secret and image scanning gates

4

Infrastructure & Kubernetes

Terraform on AWS, Helm deployment with ingress, HPA, probes, storage

5

Monitoring & GitOps

Prometheus, Grafana, alert rules, Argo CD reconciliation

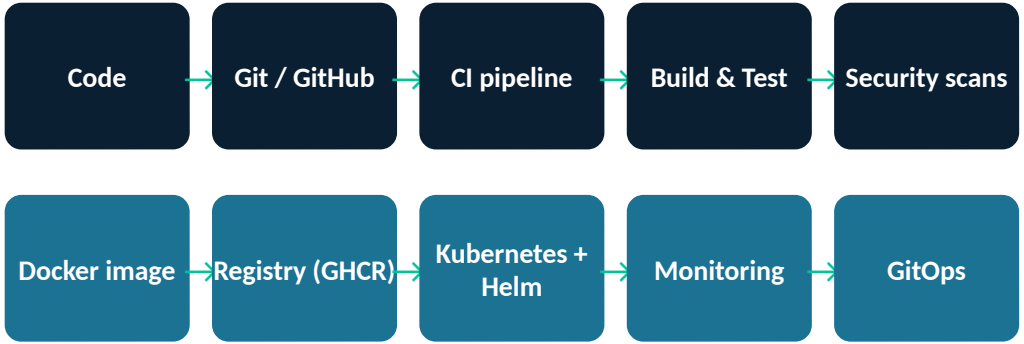
6

Troubleshooting & lessons

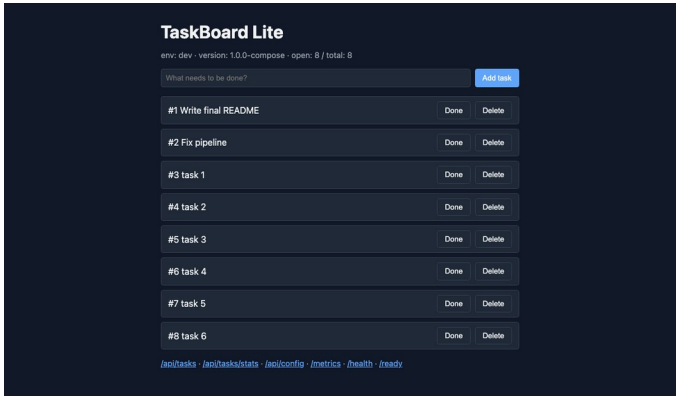
Four injected failures fixed, what the project taught me

Goal: laptop to a monitored cluster

TaskBoard Lite is a small task-management web app (Python/Flask + SQLite, Prometheus metrics). The application is deliberately simple so that the focus stays on the delivery pipeline around it.



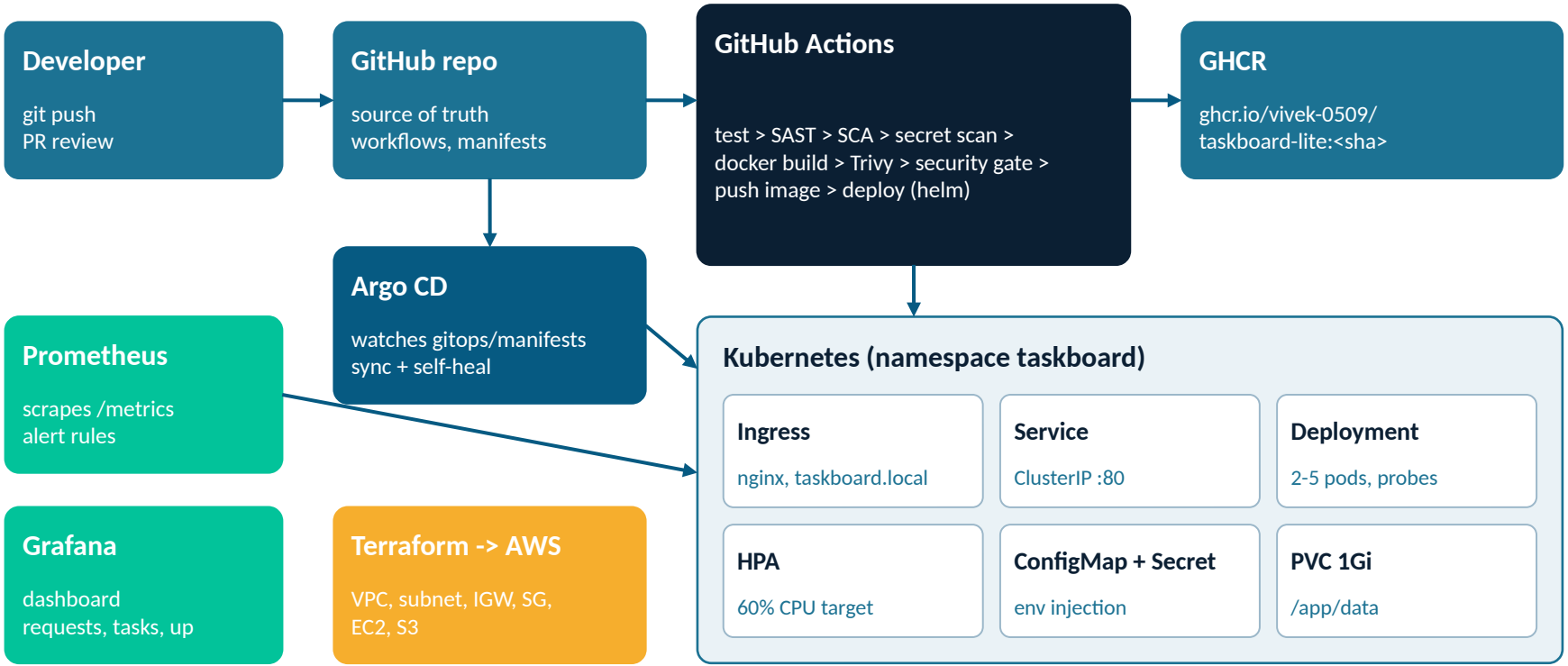
Infrastructure provisioned with Terraform on AWS (VPC, subnet, IGW, route table, security group, EC2, S3).



The running application (docker compose)

Flask 3.1 | SQLite on a PVC | /health /ready /metrics /api/tasks

Architecture



Technology stack

Application

Python 3.12, Flask 3.1, SQLite, prometheus-client, gunicorn, pytest (17 tests, 92% coverage)

Containers

Docker multi-stage build on python:3.12-alpine, non-root uid 10001, docker compose

CI/CD

GitHub Actions with 9 jobs, artifacts, GHCR login with GITHUB_TOKEN, act for local runs

DevSecOps

Bandit, pip-audit, gitleaks, Trivy, CodeQL and a security-gate job

Kubernetes + Helm

minikube 1.39 / Kubernetes 1.37, nginx ingress, metrics-server, HPA, probes, PVC, Helm chart

Infrastructure

Terraform 1.14 with the AWS provider 6.x: VPC, subnet, IGW, route table, SG, EC2, S3

Monitoring

Prometheus 3.5, Grafana 11, alert rules, ServiceMonitor and PrometheusRule manifests

GitOps

Argo CD Application with automated sync, prune and self-heal over kustomize manifests

Application, tests and container image

17

pytest tests passing

92% line coverage

0

HIGH / CRITICAL CVEs

Trivy on the final image

10001

non-root container user

read-only friendly, Alpine base

REST API

- GET /health (liveness), GET /ready (readiness, checks the database), GET /metrics (Prometheus)
- GET/POST /api/tasks, GET/PUT/DELETE /api/tasks/<id>, GET /api/tasks/stats, GET /api/config
- Configuration from environment: APP_ENV, LOG_LEVEL, FEATURE_FLAGS, DB_PASSWORD (never printed)

Docker

- Stage 1 installs dependencies; stage 2 copies only the result into python:3.12-alpine
- The Debian slim base failed the Trivy gate with 44 HIGH CVEs; switching to Alpine gave a clean scan
- docker compose runs the app with Prometheus and Grafana for local monitoring

CI/CD pipeline: GitHub Actions



Each job runs on its own ubuntu-latest runner; needs: chains them so a failure stops the pipeline. Push and deploy run only on main. The image is tagged with the Git commit SHA and handed between jobs as an artifact.

Terminal Local (2) x + v

```
vivek@Viveks-MacBook-Air devops % act -W .github/workflows/session21-final-pipeline.yml --list
Stage  Job ID           Job name                        Workflow name         Workflow file               Events
0      test             Unit tests (pytest)             Final DevOps Pipeline  session21-final-pipeline.yml  push,workflow_dispatch
1      sast             SAST (bandit)                   Final DevOps Pipeline  session21-final-pipeline.yml  push,workflow_dispatch
2      sca              SCA (pip-audit)                 Final DevOps Pipeline  session21-final-pipeline.yml  push,workflow_dispatch
3      secret-scan      Secret scanning (gitleaks)      Final DevOps Pipeline  session21-final-pipeline.yml  push,workflow_dispatch
4      docker-build     Build image                     Final DevOps Pipeline  session21-final-pipeline.yml  push,workflow_dispatch
5      image-scan       Image scan (trivy)              Final DevOps Pipeline  session21-final-pipeline.yml  push,workflow_dispatch
6      security-gate    Security gate                   Final DevOps Pipeline  session21-final-pipeline.yml  workflow_dispatch,push
7      push-image       Push image to GHCR              Final DevOps Pipeline  session21-final-pipeline.yml  push,workflow_dispatch
8      deploy-kubernetes Deploy to Kubernetes (kind + helm) Final DevOps Pipeline  session21-final-pipeline.yml  push,workflow_dispatch

vivek@Viveks-MacBook-Air devops %
```

Pipeline executed locally with act (all 7 CI jobs green)

- Artifacts: test results, coverage, bandit report, image.tar
- Secrets: GITHUB_TOKEN for GHCR, no credentials in the repo
- Deploy job: kind cluster on the runner, kind load, helm upgrade --install, curl smoke test
- act fallbacks replace GitHub-only actions locally

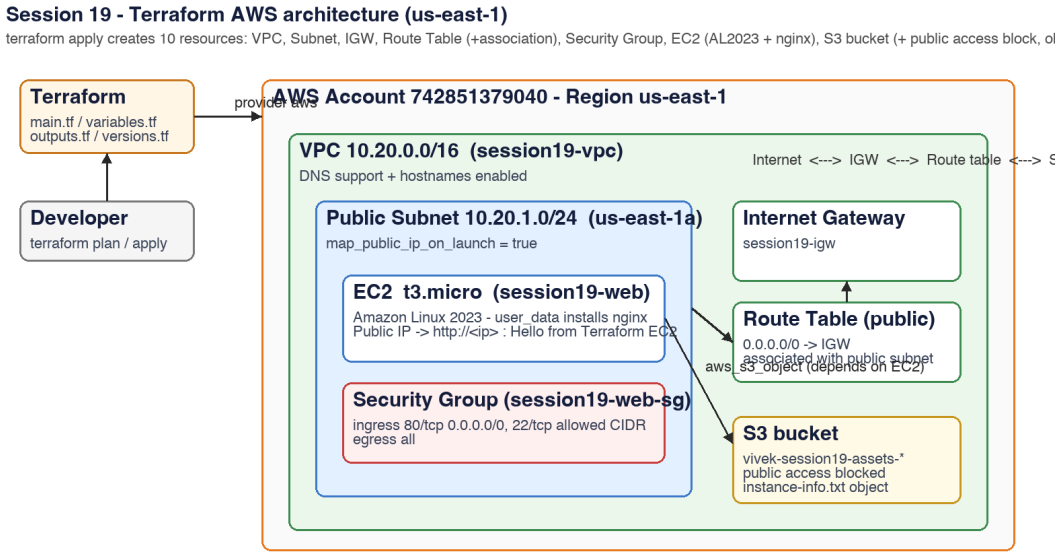
DevSecOps: gates that actually blocked

Control	Tool	What it caught / result
SAST	Bandit, CodeQL	B104 bind-all-interfaces reviewed and suppressed with a justification; 0 open findings
SCA	pip-audit --strict	Flask 3.1.1 had PYSEC-2026-2151; upgraded to 3.1.3, now 0 known CVEs
Secret scanning	gitleaks	Repository clean; a planted AWS key and GitHub token were detected in a test
Image scanning	Trivy (HIGH/CRITICAL, ignore-unfixed)	Debian base: 44 HIGH; Alpine base: 0 HIGH / 0 CRITICAL
Security gate	security-gate job (if: always())	Fails the run unless all four scans succeeded; push and deploy depend on it



Fixing real findings was part of the work: a dependency CVE, base-image CVEs and a hardening side effect (read-only root filesystem broke gunicorn until /tmp got an emptyDir).

Infrastructure as Code: Terraform on AWS



VPC 10.30.0.0/16, public subnet, Internet Gateway, route table, security group (80/22), EC2 t3.micro with Docker user_data, private S3 bucket

12

resources created by terraform apply, then destroyed

init > fmt > validate > plan > apply > destroy

full workflow executed against my AWS account

SSM lookup

latest Amazon Linux 2023 AMI, no hard-coded IDs

Kubernetes deployment with Helm

Deployment

2 replicas, startup / readiness / liveness probes, resource requests and limits, runAsNonRoot

Service + Ingress

ClusterIP behind nginx ingress at taskboard.local; verified with curl through the ingress

ConfigMap + Secret

APP_ENV, LOG_LEVEL, FEATURE_FLAGS and DB_PASSWORD injected and shown by /api/config

PVC storage

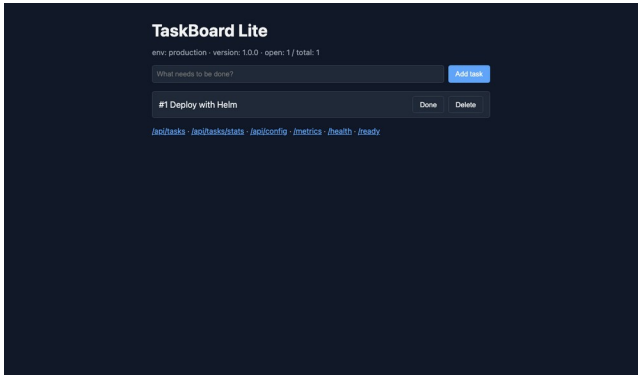
1Gi volume for the SQLite file; tasks survived a Pod deletion

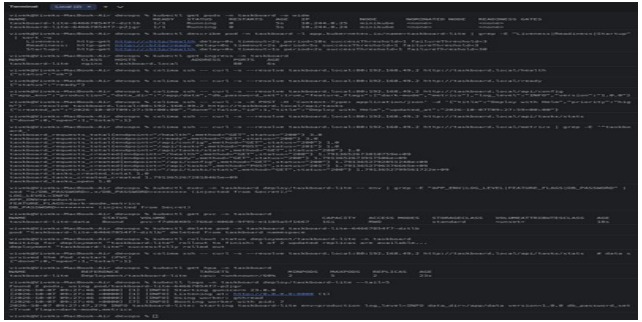
HPA

2 to 5 replicas at 60% CPU; scaled 2 -> 4 under a wget load test

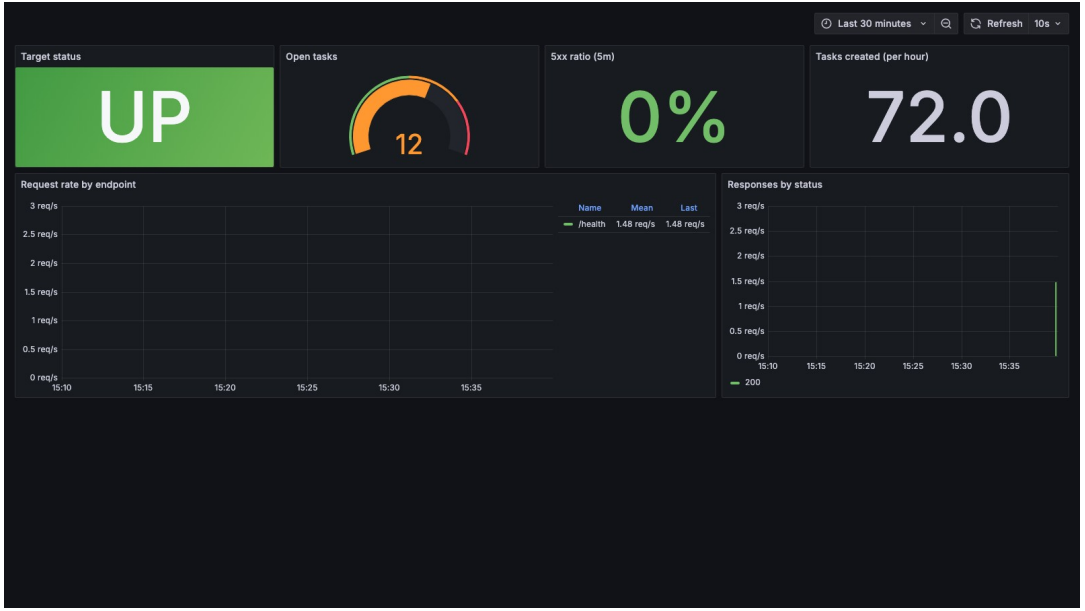
Helm chart

values.yaml / values-prod.yaml, helm lint, helm upgrade --install, used by the CD job





Monitoring and observability



Grafana dashboard provisioned from the repo: target status, open tasks, 5xx ratio, request rate by endpoint

Metrics

/metrics exposes taskboard_requests_total, taskboard_tasks_open and more; Prometheus scrapes it, kubectl top feeds the HPA

Logs

Structured application logs via kubectl logs and docker compose logs

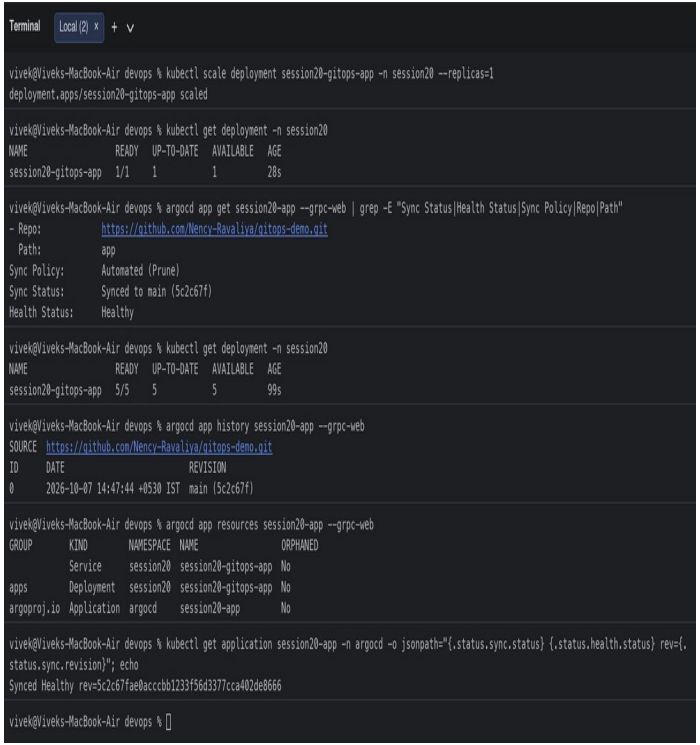
Alerts

TaskboardDown and error-rate rules; an alert was made to fire and resolve in the Session 20 demo

GitOps with Argo CD

1. Developer opens a PR with a manifest or image-tag change
2. Merge to main: Git is the only source of truth
3. Argo CD detects the new commit and syncs the namespace
4. Manual drift (kubectl scale) is reverted by selfHeal

gitops/argocd-application.yaml: automated sync, prune, selfHeal, CreateNamespace



Troubleshooting: four injected failures

#	Symptom	Investigation	Root cause and fix
1	ImagePullBackOff	kubectl describe pod: manifest unknown	Tag v9.9.9 does not exist; kubectl set image to a valid tag
2	Service has no endpoints, requests fail	kubectl get endpoints, --show-labels	Selector scenario-2 vs label scenario-02; patch the selector
3	CreateContainerConfigError	Events: couldn't find key DB_PASS	Wrong secretKeyRef key; patch it to DB_PASSWORD
4	Pods never Ready, 503 via ingress	describe pod: readiness probe 404 on /readyz	Probe path typo; patch readinessProbe path to /ready



Same loop every time: identify (kubectl get) > investigate (describe, logs, endpoints) > root cause > fix > verify. Every step is captured with before/after output in the README.

The whole course, hands-on

21
sessions completed

83
commits in the repo

62
README files

204
screenshots of real runs

Linux, shell scripting, networking, git | Docker images, networking, volumes | Kubernetes objects, services, ingress, config, storage, HPA, troubleshooting | Helm | GitHub Actions and DevSecOps | Terraform and AWS | Monitoring and GitOps

Every session folder has its own README with the commands, their real output and screenshots; the submission links point to them.

Lessons learned

A gate must bite

The dependency CVE and the base-image CVEs were real; the pipeline only became green after fixing them.

Hardening has side effects

readOnlyRootFilesystem broke gunicorn until /tmp became an emptyDir; non-root needs the image to define the user.

State shapes the strategy

A ReadWriteOnce PVC forces Recreate or a single writer; a real database belongs outside the Pod.

Test the pipeline before pushing

act runs the workflow locally; GitHub-only actions need conditional fallbacks.

Plan, apply, destroy

Terraform experiments cost nothing when every apply ends with destroy.

Git as the source of truth

Argo CD turns drift into a non-event: the cluster converges back to what is committed.



Demo and repository

Repository: github.com/vivek-0509/Devops_Final_Assignment_N_Project

Presentation video: demo/ folder (see demo/README.md for the link)

Project README: session-21-final-devops-project/final-devops-project/README.md

Thank you. Questions welcome.

Vivek Singh Solanki | 24bcs10338